# Formal Languages and Compilers

**(Linguaggi Formali e Compilatori)**

## prof. Luca Breveglieri

**(prof. A. Morzenti)**

## Written exam - 3 September 2014 - Part I: Theory

NAME:

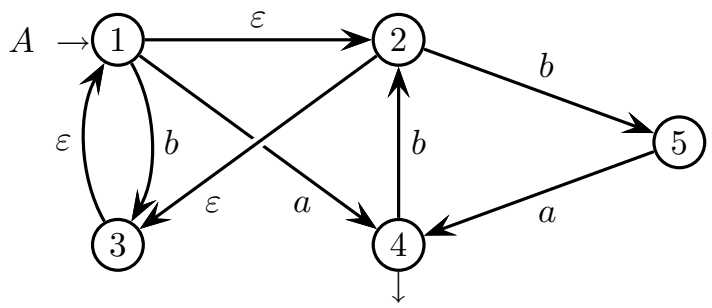
MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
  - I (80%) Theory:
    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing
    4. translation and semantic analysis
  - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

# 1 Regular Expressions and Finite Automata 20%

1. Consider the following nondeterministic finite state automaton $A$, over the alphabet $\{\, a,\ b \,\}$ and with spontaneous transitions ($\varepsilon$-transitions):




Answer the following questions:

(a) Transform the automaton $A$ into an equivalent automaton $A'$ without spontaneous transitions ($\varepsilon$-transitions), no matter if it is still nondeterministic.

(b) If the automaton $A'$ obtained before is nondeterministic, then in a systematic way of your choice transform it into an equivalent deterministic automaton $A''$, and if necessary minimize it.

(c) In a systematic way of your choice, find a regular expression $R$ that generates the (regular) language $L\,(A)$.

(d) (optional) Say if the language $L\,(A)$ is local or not, and explain why; if such a language is local, then construct the local automaton $A'''$ that recognizes it.

---

## 2 Free Grammars and Pushdown Automata 20%

1. Consider a subset $L$ of the Dyck language with round brackets ' ( ' and ' ) ', where the strings of $L$ contain an even number of open round brackets (number 0 is even).

   Sample valid strings:

   ( ( ) )  ( ( ) ( ) ) ( )  ( ) ( ( ) ( ) ( ) ) ( )

   Sample invalid strings:

   ( )  ( ( ) ) ( )  ( ( ) ( ) )  ( ) ( ( ) ) ( )

   Answer the following questions:

   (a) Write a *BNF* grammar $G$, not ambiguous, that generates the language $L$.

   (b) (optional) Draw the syntax trees of grammar $G$ for the three sample valid strings.

---

2. One wishes one modeled the fragment of a programming language, similar yet not identical to the C language, that has the variable assignment statement and the $n$-way conditional if statement (sometimes called *chained if*), with $n \geq 1$. The $n$-way if conditional has an if-then way, a number $\geq 0$ of else-if-then ways (each such way has its own condition) and an optional final else way (unconditioned). The new language closes the whole $n$-way if conditional with exactly one keyword endif.

   The following specifications apply:

   - The variable assignment statement has a variable name on the left side, the operator ' = ' in the middle and a numerical expression on the right side.
   - The numerical expression has variables, constants, round brackets, and the operators of addition ' + ', subtraction ' − ' and multiplication ' ∗ ' , with the usual precedences (addition and subtraction are lower priority than multiplication).
   - The condition of an if is a relational expression with a comparison operator ' < ', ' == ' or ' > ' between two numerical expressions (as defined before).
   - The then way of an if is mandatory, the else if and else ways are all optional.
   - There may be statement blocks (not empty and possibly nested), which are delimited by graph brackets ' { ' e ' } '; one isolated statement in a then or else way does not need to have graph brackets around itself (but it may have them).
   - Two consecutive statements in a block must be separated by a semicolon ' ; '.
   - It is forbidden to have a semicolon ' ; ' ahead of the closed graph bracket ' } ', as well as ahead of the keywords else (possibly followed by an if) and endif.

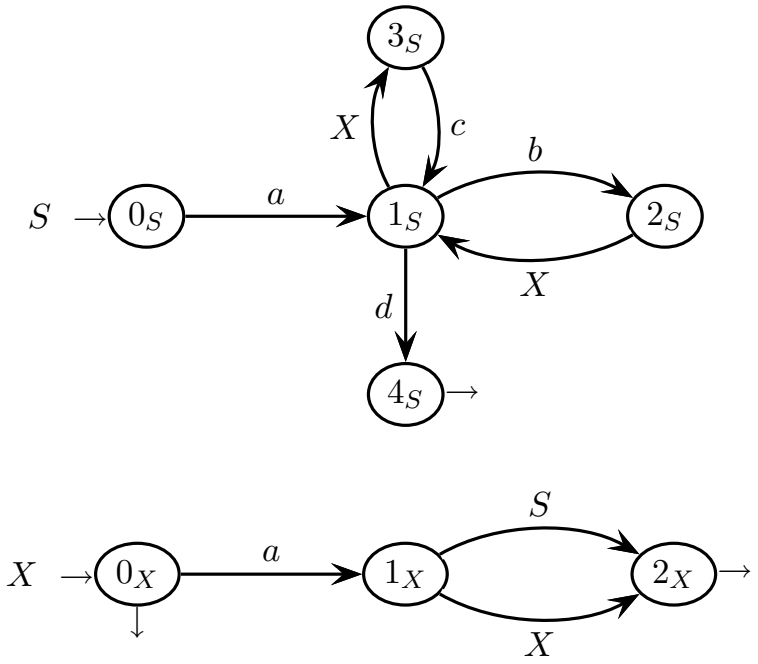
   Here are a sample 3-way if conditional and a sample 2-way (ordinary) if conditional:

```
{                                  /* language phrase      */
   a = b + c * (a + 2) ;           /* assignment           */
   {                               /* nested block         */
      a = 1 + c ;
      c = b * a
   } ;
   if a - b > c                    /* 3-way if: 1-st way   */
      b = 2
   else if 2 * (b + c) < 3         /* 3-way if: 2-nd way   */
      a = 1
   else {                          /* 3-way if: 3-rd way   */
      if b == c {                  /* 2-way if: 1-st way   */
         c = 3
      } else                       /* 2-way if: 2-nd way   */
         a = 2 * b
      endif ;                      /* end of the 2-way if  */
      c = - c                      /* unary minus sign     */
   } endif                         /* end of the 3-way if  */
}
```

   A language phrase consists of exactly one statement block. Write an *EBNF* grammar, not ambiguous, that generates the language fragment described above.

## 3 Syntax Analysis and Parsing 20%

1. Consider the following recursive machine network $\mathcal{M}$, over the terminal alphabet $\{a, b, c, d\}$ and the nonterminal alphabet $\{S, X\}$ (axiom $S$):




Answer the following questions:

(a) Construct a part of the pilot $\mathcal{P}$ of net $\mathcal{M}$ sufficient to parse the following valid string, according to the bottom-up analysis method (i.e., method *ELR*):

$a\ b\ d$

(b) By using the pilot part $\mathcal{P}$ constructed before, simulate the bottom-up parsing process of the valid string "$a\,b\,d$" considered above. You should show move-by-move the evolution of the parser stack, list one-by-one the shift and reduction moves the parser executes, and draw the syntax tree the parser builds. Please fill and complete the simulation table prepared on the next page.

(c) Examine the condition $ELL(1)$ for net $\mathcal{M}$ by computing all the guide sets on the arcs and final darts of net $\mathcal{M}$, say if the net $\mathcal{M}$ is $ELL(1)$ and explain why.

(d) (optional) The net $\mathcal{M}$ is *not* of type $ELR(1)$. Continue (if it is necessary) the construction of the pilot $\mathcal{P}$ of net $\mathcal{M}$, as much as it helps to find a conflict (which has to exist). Then find a valid string, i.e., a string of $L(\mathcal{M})$, that is *nondeterministically* recognized by pilot $\mathcal{P}$, and briefly explain why this happens.

---

simulation table of the parsing process of string "$a\,b\,d$" to be completed

(the number of rows is not significant)

| stack base | ⟶ stack contents ⟶ | | | | | | | | move executed |
|---|---|---|---|---|---|---|---|---|---|
| | empty initial stack | | | | | | | | initialization |
| | $a$ | $\langle\, 1_S,\, \dashv,\, \sharp 1 \,\rangle$ $\left\langle \textcircled{0_X}, c, \perp \right\rangle$ | | | | | | | terminal shift $I_0 \xrightarrow{a} I_1$ |
| | | | | | | | | | |
| | | | | | | | | | |
| $\langle\, 0_S,\, \dashv,\, \perp \,\rangle$ | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |
| | | | | | | | | | |

# 4 Translation and Semantic Analysis 20%

1. Consider the syntactic translation of the $n$-way if conditional with endif, for $n \geq 3$ (see also ex. 2.2), into the ordinary 2-way if conditional also with endif. The $n$-way if must be translated into a series (of length $n-1$) of 2-way if's, which are nested inside of their else ways and are closed one-by-one with as many keywords endif.

   Here is the sample translation of a 3-way if into two nested 2-way if's:

```
if cond_1       /* 1-st way */
   block_1
else if cond_2  /* 2-nd way */
   block_2
else            /* 3-rd way */
   block_3
endif
```

source with one 3-way if

```
if cond_1      /* 1-st way */
   block_1
else           /* 2-nd way */
   if cond_2   /* 1-st way */
      block_2
   else        /* 2-nd way */
      block_3
   endif
endif
```

translation into two nested 2-way if's

   The translation emits an output different from the input if the source if conditional has three or more ways. If the source if conditional is 2-way or 1-way only, then the translation outputs it unchanged. Suppose the condition and the block are generated by the terminals $C$ and $B$, respectively, which do not need to be expanded.

   Here is the source *BNF* grammar $G_s$ (axiom $S$), which generates the $n$-way if ($n \geq 1$):

$$G_s \begin{cases} S & \rightarrow & \textsf{if } C\ B\ S_{else} \\ S_{else} & \rightarrow & \textsf{else if } C\ B\ S_{else} \\ S_{else} & \rightarrow & \textsf{else } B\ \textsf{endif} \\ S_{else} & \rightarrow & \textsf{endif} \end{cases}$$

   Answer the following questions:

   (a) Write the destination grammar $G_d$ that corresponds to the source grammar $G_s$, for the translation described above (the translation must work for every $n \geq 1$).

   (b) Draw the source and destination syntax trees of the sample translation shown above (consider the symbols $C$ and $B$ as terminals).

   (c) (optional) Examine the translation scheme (or grammar) written before, say if it is deterministic or not, and briefly explain why.

---

2. A grammar $G$ generates expressions that consist of a non-empty list delimited by round brackets '(' and')'. Such a list contains one or more elements of these two types: an atom represented by terminal $a$ or a non-empty sublist delimited by brackets (as before); and so on recursively down to an arbitrary sublist nesting depth.

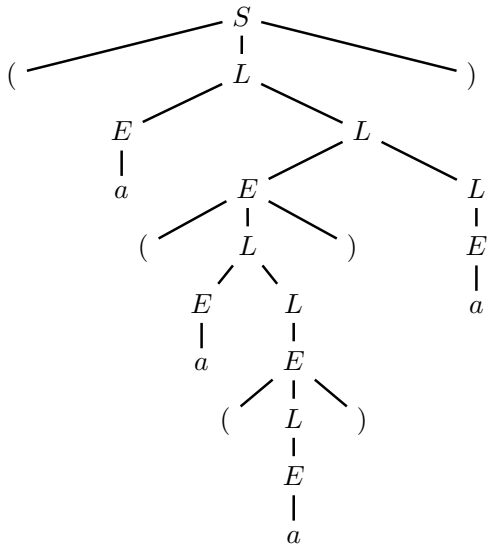
   The nesting level of an element (atom or sublist) is the number of bracket pairs that enclose the element. Here is a sample expression $e$:

   $$e = \Big( a \; \big( a \; ( a ) \big) \; a \Big)$$

   The expression $e$ has three elements at level 1, i.e., the $1^{st}$ and $4^{th}$ atom $a$, and the sublist $\big( a \; ( a ) \big)$; it has two at level 2, i.e., the $2^{nd}$ atom $a$ and the sublist $( a )$; and only one at level 3, i.e., the $3^{rd}$ atom $a$. It has a *total* number of elements $3+2+1 = 6$. The sublist $\big( a \; ( a ) \big)$ has three elements in total: two atoms $a$ and the sublist $( a )$.

   Here is the grammar $G$ (axiom $S$) of the expressions, and the syntax tree of $e$:

$$G \begin{cases} S & \rightarrow & \text{`(' } L \text{ `)'} \\ L & \rightarrow & E \; L \\ L & \rightarrow & E \\ E & \rightarrow & a \\ E & \rightarrow & \text{`(' } L \text{ `)'} \end{cases}$$




   Answer the following questions (use the tables and trees on the next pages):

   (a) Write an attribute grammar $G_a$ based on the syntactic support $G$. Grammar $G_a$ computes an integer attribute $n \geq 1$ that expresses the total number of elements (atoms and sublists) in the expression (i.e., the number of nonterminals $E$). In the tree root of $e$ it holds $n = 6$. Decorate the tree of $e$ with the values of $n$.

   (b) By means of an integer attribute $d \geq 1$, associate to each element (atom or sublist) the respective nesting level. Decorate the tree of $e$ with the values of $d$.

   (c) (optional) By means of a boolean attribute $v$, and possibly of more ones if they help, verify if in the expression there is a proper sublist (i.e., not coincident with the entire expression) that has a total number of elements equal to its nesting level. If there is, in the tree root it holds $v = T$, otherwise it holds $v = F$. The expression $e$ has $v = F$ in the root. Instead, the expression $e' = \big( a \; ( a ) \; a \big)$ (different from $e$) has $v = T$, because its proper sublist $( a )$ has only one element and is at level 1. Decorate the tree of $e$ with the attribute values.

attributes already assigned to be used for grammar $G_a$

write the field *type*

| *type* | *name* | *domain* | *nonterm.* | *meaning* |
|---|---|---|---|---|
| | $n$ | integer $\geq 1$ | $S$, $L$, $E$ | total number of elements (atoms and sublists) |
| | $d$ | integer $\geq 1$ | $L$, $E$ | nesting level of an element (atom or sublist) |
| | $v$ | boolean | $S$, $L$, $E$ | this predicate is true if and only if in the expression there is a proper sublist (i.e., not coincident with the entire expression) that has a total number of elements (atoms and sublists) equal to its nesting level |

possible auxiliary attributes to be added for question $c$ (if they help)

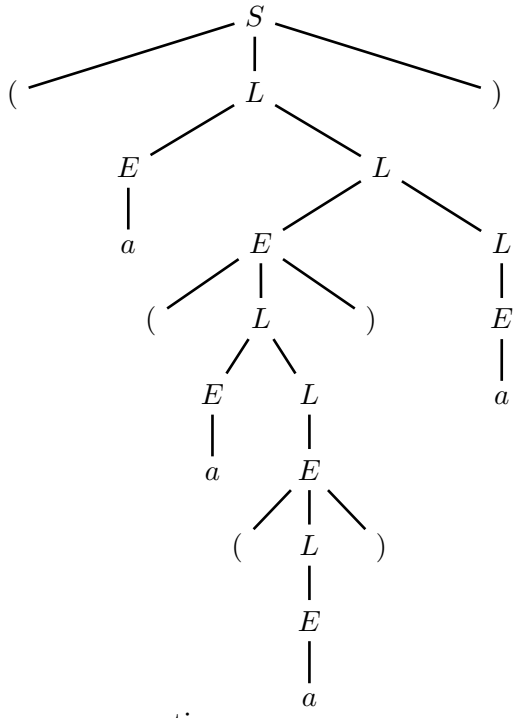
| *type* | *name* | *domain* | *nonterm.* | *meaning* |
|---|---|---|---|---|
| | | | | |

| # | *syntax* | *semantics* - question $a$ |
|---|---|---|
| | | $\mathbf{n_0} = \mathbf{n_1}$ |
| 1: | $S_0 \rightarrow ( L_1 )$ | |
| 2: | $L_0 \rightarrow E_1 L_2$ | |
| 3: | $L_0 \rightarrow E_1$ | |
| 4: | $E_0 \rightarrow a$ | |
| 5: | $E_0 \rightarrow ( L_1 )$ | |

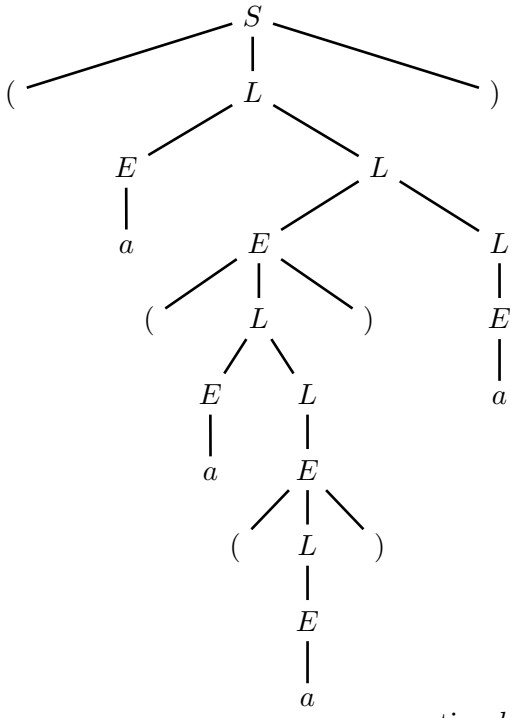
| # | *syntax* | *semantics* - question $b$ |
|---|---|---|
| | | $\mathbf{d_1 = 1}$ |
| 1: | $S_0 \rightarrow ( L_1 )$ | |
| 2: | $L_0 \rightarrow E_1 \; L_2$ | |
| 3: | $L_0 \rightarrow E_1$ | |
| 4: | $E_0 \rightarrow a$ | |
| 5: | $E_0 \rightarrow ( L_1 )$ | |

| # | *syntax* | *semantics* - question $c$ |
|---|---|---|
| | | $\mathbf{v_0} = \mathbf{v_1}$ |
| 1: | $S_0 \rightarrow ( L_1 )$ | |
| 2: | $L_0 \rightarrow E_1 L_2$ | |
| 3: | $L_0 \rightarrow E_1$ | |
| 4: | $E_0 \rightarrow a$ | |
| 5: | $E_0 \rightarrow ( L_1 )$ | |

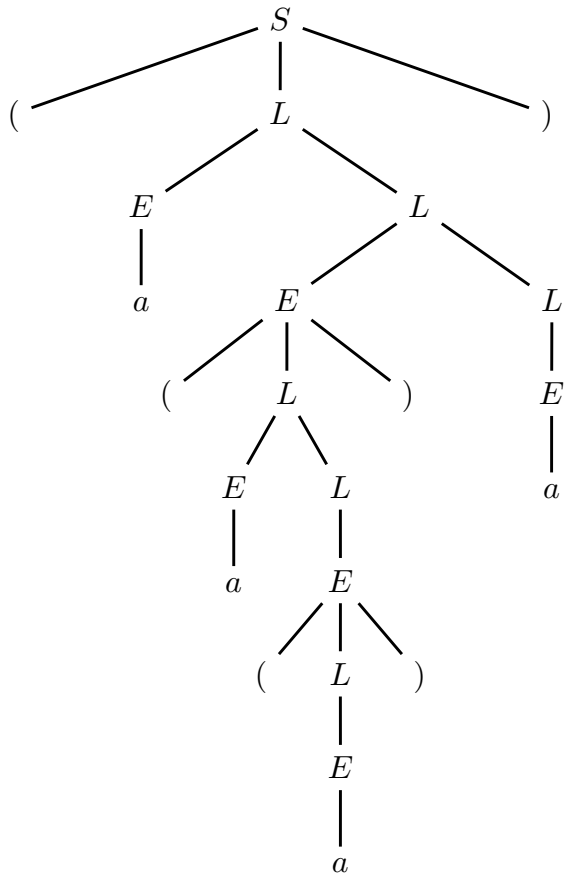
syntax trees to be decorated (one for each question)




question $a$




question $b$




question $c$